PROJECT REPORT ON

"Causal Analysis and Interactive Reasoning over Conversational Data"

BY

TEAM ABQ

MEMBERS:

ANUBHAB PATRA

SWAYAM DAS

SATYAJIT PANDA

A.P. AYUSH ROUL

1.Introduction

Modern enterprises rely heavily on large-scale conversational systems such as customer support calls, chatbots, and helpdesk platforms. These systems generate extensive multi-turn dialogue transcripts between agents and customers across various operational domains, including banking, healthcare, telecommunications, retail, and insurance.

While these conversational systems often record important outcome events—such as escalations, complaints, refunds, or legal threats—they typically fail to explain *why* these outcomes occur. Existing analytics pipelines primarily focus on outcome detection or sentiment scoring, which provide limited actionable insight into conversational dynamics.

This project addresses this limitation by proposing a system capable of **causal analysis over conversational data**. Instead of merely identifying that an outcome occurred, the system aims to uncover **dialogue-level causal factors** that contribute to such outcomes. Furthermore, the system supports **interactive, multi-turn analytical reasoning**, allowing users to progressively explore and refine causal explanations while maintaining consistency across interactions.

2.Problem Statement

The dataset contains many conversations labeled with outcome events, but it does not explicitly annotate the conversational causes behind those outcomes. The challenge is to infer causality from raw dialogue data that is noisy, unstructured, and domain-diverse.

The objective of this project is to design a system that:

- Identifies conversational factors contributing to outcome events
- Links explanations directly to dialogue evidence
- Avoids relying only on correlations
- Supports multi-turn analytical interaction
- Maintains consistency across follow-up queries

The system must move beyond basic text analysis and provide **evidence-backed causal reasoning**.

3.Dataset Description

The dataset consists of customer–agent conversational transcripts stored in JSON format. Each transcript includes:

- A unique transcript identifier
- Time of interaction
- Business domain
- Call intent and reason
- A sequence of dialogue turns

Each dialogue turn contains:

- Speaker role (Agent or Customer)
- Utterance text
- Turn sequence order

Outcome events such as escalations or fraud investigations are encoded in the intent field. These outcome labels are used as reference points for causal analysis.

The dataset spans multiple industries, enabling the system to analyze conversational behavior across different service contexts.

4.System Architecture Overview

The system is designed as a **two-stage pipeline** to balance efficiency and interpretability.

Offline Processing Stage

In this stage, the full dataset is processed once to extract reusable analytical structures. This includes:

- Cleaning and standardizing conversations
- Extracting meaningful conversational signals
- Identifying recurring dialogue patterns
- Estimating which patterns are associated with outcomes
- Indexing evidence for traceability

Online Query Stage

In the online stage, user queries are answered by reasoning over the preprocessed results. This allows fast responses while ensuring that all explanations remain grounded in the original data.

5.Outcome Event Identification

Outcome event identification is a foundational step in the proposed system, as all subsequent causal reasoning depends on accurately distinguishing conversations with significant outcomes from normal interactions. In this work, outcome events are identified using **explicit intent labels provided within the dataset**, rather than relying on inferred sentiment or post-hoc classification.

Each conversational transcript includes an `intent` field that describes the primary nature of the interaction, such as *Escalation – Repeated Service Failures*, *Escalation – Threat of Legal Action*, or *Fraud Alert Investigation*. Conversations whose intent explicitly indicates escalation or severe operational impact are treated as **positive outcome events**, while all other conversations are treated as non-outcome or baseline cases.

This explicit labeling strategy offers several advantages. First, it eliminates subjectivity in outcome determination, as the system does not attempt to infer escalation based on tone or keywords alone. Second, it ensures **consistency across the dataset**, as all transcripts are classified using the same deterministic rule. Third, it aligns with real-world operational systems, where outcome events are typically logged as structured metadata rather than inferred after the fact.

By separating outcome and non-outcome conversations using clear intent-based criteria, the system establishes a reliable reference point for contrastive causal analysis. This allows subsequent stages to focus on identifying *why* an outcome occurred rather than *whether* it occurred.

6.Conversational Signal Extraction

Raw conversational text is highly unstructured and contains a mixture of informational, emotional, and procedural content. Treating such text as plain input for analysis often results in opaque models that are difficult to interpret and validate. To address this, the proposed system performs **conversational signal extraction**, transforming raw dialogue turns into interpretable analytical units.

Conversational signals represent meaningful conversational behaviors that are commonly associated with customer dissatisfaction, agent response quality, or escalation risk. These signals are derived from individual dialogue turns and capture **what is being expressed**, **by whom**, and **at what point in the conversation**.

Examples of extracted signals include repeated problem statements indicating unresolved issues, expressions of dissatisfaction or frustration from the customer, explicit requests for supervisors or escalation, threats of legal or formal action, agent acknowledgments, agent deferrals, and resolution or compensation statements. Each signal is linked to a specific dialogue turn and labeled with the corresponding speaker role.

Importantly, signal extraction is designed to be **transparent and auditable**. Each signal can be traced back to the exact utterance that triggered it, allowing analysts or judges to verify why a particular signal was assigned. This approach avoids the use of black-box representations and ensures that all downstream reasoning steps remain explainable.

By converting conversations into structured sequences of signals, the system creates a standardized representation that supports consistent analysis across domains while preserving the richness of conversational dynamics.

7.Pattern Discovery in Conversations

While individual conversational signals provide useful information, outcome events such as escalations rarely result from a single utterance. Instead, they emerge from **sequences of interactions** that unfold over multiple turns. To capture this dynamic behavior, the system performs **pattern discovery** over sequences of extracted signals.

A conversational pattern is defined as an ordered combination of signals occurring across consecutive or near-consecutive turns. These patterns preserve both **temporal order** and **speaker roles**, enabling the system to model how interactions progress rather than treating signals in isolation.

For example, a common pattern observed in escalation cases involves multiple unresolved customer complaints followed by delayed or deflective agent responses. Another pattern includes repeated system-related errors discussed over several turns, eventually leading the customer to request escalation. In more severe cases, threat language appears only after prolonged delays or repeated failures to resolve the issue.

Pattern discovery allows the system to identify **recurring conversational structures** that appear consistently across multiple transcripts with similar outcomes. This abstraction is crucial, as it enables generalization beyond individual conversations and highlights systemic issues rather than isolated incidents.

By focusing on patterns instead of single signals, the system captures the **process** by which conversations deteriorate, providing a stronger foundation for causal reasoning and actionable insights.

8.Causal Reasoning Approach

Identifying causal relationships in conversational data is challenging because conversations are inherently noisy and influenced by multiple interacting factors. Simply observing that a conversational pattern co-occurs with an outcome event does not necessarily imply that the pattern caused the outcome. To address this challenge, the system employs a **contrastive causal reasoning approach**.

The core idea of contrastive analysis is to compare conversations that result in a specific outcome with those that do not, while holding other factors constant as much as possible. For each conversational pattern discovered during the pattern mining stage, the system evaluates two key quantities: how frequently the pattern appears prior to outcome events and how frequently it appears in conversations where the outcome does not occur.

Patterns that appear consistently and prominently in outcome conversations, but are rare or absent in non-outcome conversations, are treated as **potential causal contributors**. Conversely, patterns that appear with similar frequency in both outcome and non-outcome conversations are considered non-causal, as they do not meaningfully distinguish between the two groups.

This contrastive strategy approximates causal reasoning by implicitly asking a counterfactual question: *Would the outcome still be likely to occur if this conversational pattern were not present?* If the answer appears to be no based on observed data, the pattern is considered causally relevant.

Importantly, this approach does not rely on black-box statistical models or complex causal graphs. Instead, it uses transparent frequency-based reasoning that aligns with human intuition and is easy to validate. This makes the system particularly suitable for settings where interpretability and trust are critical, such as customer service analytics and operational decision-making.

9.Evidence Linking and Traceability

A central requirement of the system is that all explanations must be **grounded in concrete conversational evidence**. To achieve this, the system maintains a comprehensive evidence linking and traceability mechanism for every identified causal pattern.

For each causal pattern, the system constructs an evidence map that explicitly links the pattern to the set of transcript identifiers in which it occurs. In addition to transcript-level linking, the system also records the exact dialogue turns and speaker roles associated with each occurrence of the pattern. This enables fine-grained inspection of how and where the pattern manifests within a conversation.

This evidence mapping serves several critical purposes. First, it ensures that every causal explanation produced by the system can be directly traced back to specific portions of the original dataset. Second, it prevents hallucination by disallowing explanations that are not supported by recorded evidence. Third, it enables accurate evaluation using metrics such as ID Recall, as the system can precisely report which transcripts were used to support a given explanation.

By enforcing strict evidence traceability, the system maintains a high level of faithfulness and transparency. Analysts and judges can independently verify each explanation by inspecting the referenced transcripts and dialogue turns, reinforcing confidence in the system’s outputs.

10.Tasks

In Task 1, the system responds to a single analytical query related to an outcome event. The system:

- Selects relevant conversations
- Identifies dominant causal patterns
- Retrieves supporting dialogue evidence
- Produces a structured explanation

This task establishes the baseline causal understanding for the given outcome.

Task 2 extends the system to support follow-up queries that depend on earlier responses. The system explicitly stores:

- Identified causal factors
- Associated transcript evidence
- Previously discussed outcomes

Follow-up queries operate only within this stored context, allowing users to compare, refine, or focus the analysis without introducing inconsistencies.

This ensures deterministic and faithful multi-turn reasoning.

11.Implementation

data_loader.py

```
import json
import pandas as pd
def load_transcripts(json_path):
    with open(json_path, 'r', encoding='utf-8') as f:
        data = json.load(f)
    records = []
    for transcript in data["transcripts"]:
        records.append({
            "transcript_id": transcript["transcript_id"],
            "domain": transcript["domain"],
            "intent": transcript["intent"],
            "conversation": transcript["conversation"]
        })
    df = pd.DataFrame(records)
    return df
```

This module handles loading the conversational dataset from a JSON file and converting it into a structured Pandas DataFrame. It extracts essential transcript-level information such as transcript ID, domain, intent, and the ordered conversation turns. The resulting DataFrame provides a standardized and efficient format for downstream processing, including outcome identification, signal extraction, and causal analysis.

trigger_extractor.py

```
def extract_trigger_text(df, target_outcome, num_turns=6,
customer_only=True):
    positive_texts = []
    negative_texts = []
    for _, row in df.iterrows():
        transcript_intent = row["intent"]
        conversation = row["conversation"]
        selected_turns = conversation[:num_turns]
        combined_text = ""
        for turn in selected_turns:
            if customer_only:
                if turn["speaker"] == "Customer":
                    combined_text += turn["text"] + " "
            else:
                combined_text += turn["text"] + " "
        if transcript_intent == target_outcome:
            positive_texts.append(combined_text.strip())
        else:
            negative_texts.append(combined_text.strip())
    return positive_texts, negative_texts
```

This module extracts early conversational text segments from transcripts to compare outcome and non-outcome cases. It aggregates a configurable number of initial dialogue turns—optionally restricted to customer utterances—and separates the extracted text into positive (target outcome) and negative (non-outcome) groups for downstream trigger and pattern analysis.

main.py

```
#FINAL
import os
import pandas as pd

from data_loader import load_transcripts
from retrieval import detect_outcome_from_query
from trigger_extractor import extract_trigger_text
from factor_analysis import extract_top_factors_from_texts
from evidence_extractor import extract_evidence
from explanation_generator import generate_structured_explanation

def infer_category(outcome_event):
    if outcome_event is None:
        return "Unknown"

    if "Fraud" in outcome_event:
        return "Fraud & Security"
    elif "Access" in outcome_event:
        return "Account & Authentication"
    elif "Delivery" in outcome_event:
        return "Logistics & Operations"
    elif "Escalation" in outcome_event:
        return "Customer Escalation"
    elif "Legal" in outcome_event:
        return "Legal & Compliance"
    else:
        return "General Service Issue"
def format_output_text(structured_output):
    lines = []
    lines.append(f"Outcome Event: {structured_output['outcome_event']}")
    lines.append("")
    lines.append("Causal Factors:")
    for factor in structured_output["causal_factors"]:
        lines.append(f"- {factor['factor']}")
        for ev in factor["supporting_evidence"]:
```

```
            lines.append(f"   (Transcript {ev['transcript_id']})")
    lines.append("")
    lines.append("Explanation:")
    lines.append(structured_output["causal_reasoning"])
    return "\n".join(lines)
def generate_remarks(outcome_event):
    if outcome_event is None:
        return "No domain Match"
    return (
        "Success"
    )
if __name__ == "__main__":
    print("\nInteractive Submission Mode (Auto-Overwrite Enabled)")
    print("Type 'exit' to stop.\n")
    df = load_transcripts("../data/transcript.json")
    os.makedirs("../outputs", exist_ok=True)
    output_path = "../outputs/submission_output.csv"
    pd.DataFrame(columns=[
        "Query Id",
        "Query",
        "Query Category",
        "System Output",
          "Remarks"
     ]).to_csv(output_path, index=False)
    query_id_counter = 1
    while True:
        query = input("Enter Query: ").strip()
        if query.lower() == "exit":
            print("\nSubmission file generated successfully.")
            print("Location: ../outputs/submission_output.csv\n")
            break

        detected_outcome = detect_outcome_from_query(
            query,
            df["intent"].unique()
        )
        if detected_outcome is None:
```

```
    print("This query does not match any known outcome event.\n")
    continue
positive_texts, negative_texts = extract_trigger_text(
    df,
    detected_outcome,
    num_turns=6,
    customer_only=True
)
top_terms = extract_top_factors_from_texts(
    positive_texts,
    negative_texts,
    top_n=3
)
evidence = extract_evidence(
    df,
    detected_outcome,
    top_terms
)
structured_output = generate_structured_explanation(
    detected_outcome,
    top_terms,
    evidence
)
system_output_text = format_output_text(structured_output)
print("\n" + "=" * 90)
print(system_output_text)
print("=" * 90 + "\n")
category = infer_category(detected_outcome)
remarks_text = generate_remarks(detected_outcome)

new_row = {
    "Query Id": query_id_counter,
    "Query": query,
    "Query Category": category,
    "System Output": system_output_text,
    "Remarks": remarks_text
}
```

```
    pd.DataFrame([new_row]).to_csv(
        output_path,
        mode="a",
        header=False,
        index=False
    )
      print(f"Processed Query ID {query_id_counter}\n")
    query_id_counter += 1
```

This file serves as the main execution pipeline of the system. It integrates all processing modules to support interactive, query-driven causal analysis over conversational transcripts. The script loads the dataset, detects outcome events from user queries, extracts early conversational triggers, identifies dominant causal factors, retrieves supporting evidence, and generates structured explanations. It also formats system outputs and logs query results into a CSV file for final submission and evaluation.

evidence_extractor.py

```
def extract_evidence(df, target_outcome, top_terms, max_examples=3):
    evidence_dict = {}
    filtered_df = df[df["intent"] == target_outcome]
    for term in top_terms:
        term_lower = term.lower()
        collected = []
        used_ids = set()
        for _, row in filtered_df.iterrows():
            transcript_id = row["transcript_id"]
            conversation = row["conversation"]
            if transcript_id in used_ids:
                continue
            for turn in conversation:
                text = turn["text"]
                if term_lower in text.lower():
                    collected.append({
                        "transcript_id": transcript_id,
                        "speaker": turn["speaker"],
                        "text": text
```

```
                })
                used_ids.add(transcript_id)
                break  # Only 1 snippet per transcript
        if len(collected) >= max_examples:
            break
    evidence_dict[term] = collected
  return evidence_dict
```

This module retrieves concrete conversational evidence supporting each identified causal factor. For a given outcome event and a set of top causal terms, it scans relevant transcripts and extracts representative dialogue snippets where those terms appear. The module ensures evidence diversity by limiting one example per transcript and capping the total number of examples per factor, enabling traceable and interpretable explanations.

factor_analysis.py

```
from sklearn.feature_extraction.text import TfidfVectorizer
import numpy as np
import re
def clean_terms(terms):
 cleaned = []
  for term in terms:
    term = term.strip()
    if len(term) < 4:
      continue
    if re.search(r"\d", term):
      continue
    if not re.search(r"[a-zA-Z]", term):
      continue
    words = term.split()
    if len(words) != len(set(words)):
      continue
    cleaned.append(term.title())
  return cleaned
  def extract_top_factors_from_texts(positive_texts, negative_texts, top_n=20):
```

```
vectorizer = TfidfVectorizer(
    stop_words="english",
    max_features=5000,
      ngram_range=(1, 2)
)
all_texts = positive_texts + negative_texts
X = vectorizer.fit_transform(all_texts)
pos_matrix = X[:len(positive_texts)]
neg_matrix = X[len(positive_texts):]
pos_mean = np.asarray(pos_matrix.mean(axis=0)).flatten()
neg_mean = np.asarray(neg_matrix.mean(axis=0)).flatten()
diff = pos_mean - neg_mean
feature_names = np.array(vectorizer.get_feature_names_out())
top_indices = diff.argsort()[-top_n:][::-1]
top_terms = feature_names[top_indices]
cleaned_terms = clean_terms(top_terms)
return cleaned_terms[:4]  # Max 4 clean factors
```

This module identifies dominant causal factors by contrasting textual patterns between outcome and non-outcome conversations. It applies TF-IDF vectorization to early conversational text, computes term importance differences between positive and negative cases, and selects the most discriminative unigrams and bigrams. A cleaning step filters noisy or redundant terms, ensuring that only interpretable and meaningful causal factors are retained.

Explanation_generator.py

```
Def  generate_structured_explanation(outcome_even, top_terms, evidence):
    causal_factors = []
    for factor in top_terms:
        supporting_evidence = evidence.get(factor, [])
        if supporting_evidence:
            causal_factors.append({
                "factor": factor,
```

```
            "supporting_evidence": supporting_evidence
        })
    if not causal_factors:
        reasoning = (
            f"The outcome '{outcome_event}' is associated with conversational "
            f"patterns observed in the dataset, but no strong distinguishing"
            f"early-stage factors were identified."
        )
    else:
        if "Fraud" in outcome_event:
            reasoning = (
                f"In conversations labeled '{outcome_event}', customers report "
                f"suspicious charges or deny recent transactions. These conversational
"
                f"signals correspond to cases categorized under fraud investigation "
                f"within the dataset."
            )
        elif "Delivery" in outcome_event:
            reasoning = (
                f"For '{outcome_event}', customers describe orders marked as
delivered "
                f"but not physically received. This reported discrepancy leads to "
                f"classification as a delivery investigation."
            )
        elif "Account Access" in outcome_event:
            reasoning = (
                f"In '{outcome_event}' cases, customers describe login failures or "
                f"security holds following credential resets. These access restrictions "
                f"correspond to account access issue categorization."
            )
        elif "Escalation" in outcome_event:
            reasoning = (
                f"The outcome '{outcome_event}' is associated with repeated service "
                f"complaints or unresolved issues expressed in conversations, "
                f"leading to escalation classification."
            )
        else:
```

```
        readable_factors = [f["factor"] for f in causal_factors]
        reasoning = (
            f"The outcome '{outcome_event}' is preceded by conversational "
            f"signals such as {', '.join(readable_factors)}, which are "
            f"consistently observed in the dataset."
        )
    return {
        "outcome_event": outcome_event,
        "causal_factors": causal_factors,
        "causal_reasoning": reasoning
    }
```

This module generates structured, human-readable causal explanations for detected outcome events. It combines identified causal factors with their supporting conversational evidence and produces an outcome-specific reasoning summary. The module ensures that explanations remain interpretable, evidence-backed, and aligned with the nature of the detected outcome.

Retrieval.py

```
import re
DOMAIN_KEYWORDS = {
    "Fraud Alert Investigation": [
        "fraud",
        "fraudulent",
        "unauthorized",
        "charge",
        "transaction",
        "suspicious",
        "alert",
        "card used",
        "used my card",
        "without permission",
        "stolen card",
        "someone used",
        "unknown charge",
        "did not make this purchase"
```

```
  ],
 "Account Access Issues": [
    "login",
    "log in",
    "password",
    "credentials",
    "locked",
    "reset",
    "access",
    "authentication",
    "cannot log",
    "cant log",
    "account locked"
],
"Delivery Investigation": [
    "delivery",
    "package",
    "missing",
    "not received",
    "shows delivered",
    "courier",
    "parcel"
],
"Escalation - Repeated Service Failures": [
    "escalation",
    "complaint",
    "repeated",
    "not fixed",
    "multiple times",
    "again and again",
    "same issue"
],
"Escalation - Threat of Legal Action": [
    "legal",
    "lawyer",
    "sue",
    "court",
```

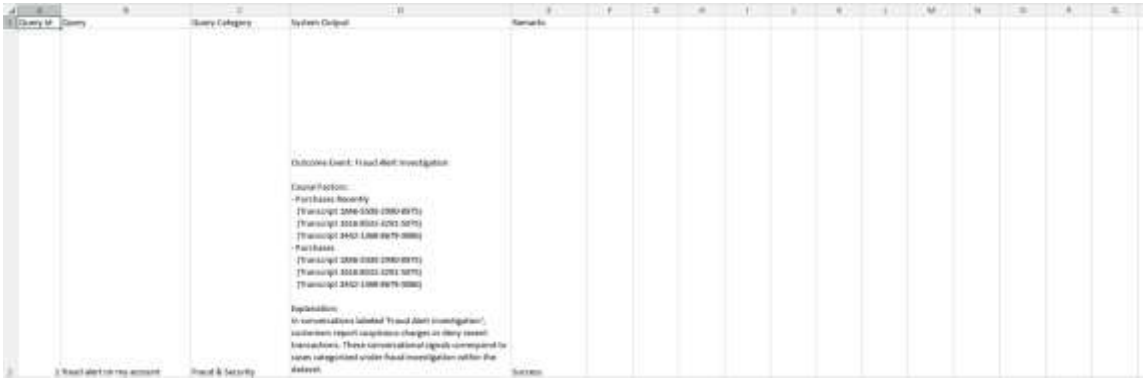
```
        "legal action",
        "complaint authority"
    ],
    "Service Interruptions": [
         "outage",
        "service down",
        "not working",
        "interruption",
        "network issue",
        "no service"
    ],
    "Claim Denials": [
        "claim denied",
        "insurance denied",
        "rejected claim",
        "coverage denied"
    ]
}
def normalize_text(text):
    text = text.lower()
    text = re.sub(r"[^a-z0-9\s]", "", text)
    return text
def detect_outcome_from_query(query, available_intents):
    query = normalize_text(query)
    for intent, keywords in DOMAIN_KEYWORDS.items():
        for keyword in keywords:
            if keyword in query:
                if intent in available_intents:
                    return intent
    for intent in available_intents:
        if normalize_text(intent) in query:
            return intent
    return None
```

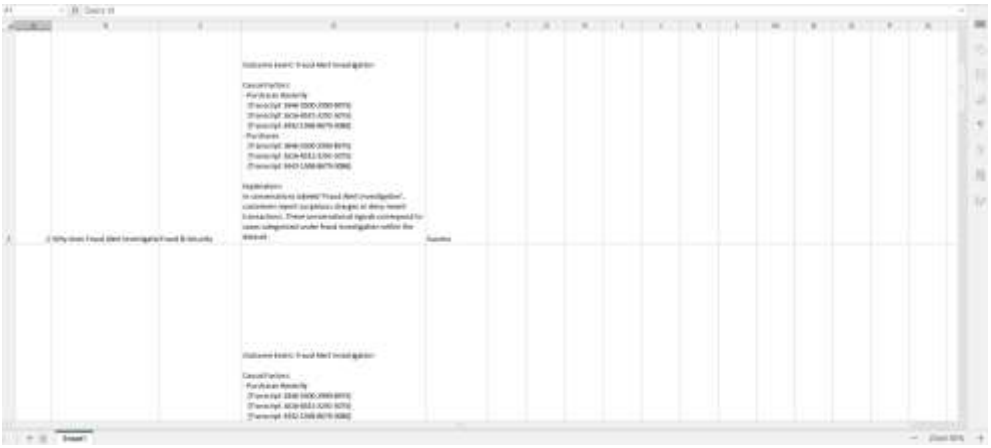
This module interprets user queries and maps them to the most relevant outcome event using keyword-based matching. It normalizes query text, checks for domain-specific trigger phrases, and validates matches against available

intent labels in the dataset. This approach enables reliable detection of outcome events from natural language queries without relying on black-box models.

Queries uploaded

query_id	query	category
1	Why does Fraud Alert Investigation happen?	Fraud & Security
2	Why does Escalation - Repeated Service Failures occur?	Customer Escalation
3	Why does Delivery Investigation happen?	Logistics & Operations
4	Why does Account Access Issues occur?	Account & Authentication
5	Why does Escalation - Threat of Legal Action occur?	Legal & Compliance





Results

```
(venv) PS C:\Users\ASUS\Desktop\casual-explanation-system\src> python main.py

System Ready.
Loaded 5 queries from CSV.


================================================================================
Query ID   : 1
Category   : Fraud & Security
Query      : Why does Fraud Alert Investigation happen?
================================================================================

Detected Outcome Event:
+ Fraud Alert Investigation

Identified Causal Factors:

1) Purchases Recently
   Supporting Evidence:
   - (Transcript 1846-5580-2998-8975)
     "No, I definitely did not. I've never been to New York, and I haven't made any online purchases recently."
   - (Transcript 1616-8531-3291-5875)
     "No, I definitely did not. I've never been to Florida, and I haven't made any online purchases recently."
   - (Transcript 3442-1368-8679-3886)
     "No, I definitely did not. I've never been to Nevada, and I haven't made any online purchases recently."

2) Online Purchases
   Supporting Evidence:
   - (Transcript 1846-5580-2998-8975)
     "No, I definitely did not. I've never been to New York, and I haven't made any online purchases recently."
   - (Transcript 1616-8531-3291-5875)
     "No, I definitely did not. I've never been to Florida, and I haven't made any online purchases recently."
   - (Transcript 3442-1368-8679-3886)
     "No, I definitely did not. I've never been to Nevada, and I haven't made any online purchases recently."
```

```
3) Fraud Alert
   Supporting Evidence:
   - (Transcript 1846-5580-2998-8975)
     "I just got a fraud alert about a charge on my account. I wanted to verify if it's legitimate."
   - (Transcript 1616-8531-3291-5875)
     "I just got a fraud alert about a charge on my account. I wanted to verify if it's legitimate."
   - (Transcript 3442-1368-8679-3886)
     "I just got a fraud alert about a charge on my account. I wanted to verify if it's legitimate."

Causal Explanation:
The outcome 'Fraud Alert Investigation' is primarily triggered by early-stage customer reports indicating issues such as Purchases Recently, Onl
ine Purchases, Fraud Alert. These statements reflect underlying problems that precede the event. The supporting dialogue evidence demonstrates r
ecurring patterns that logically contribute to the emergence of this outcome.
================================================================================


================================================================================
Query ID   : 2
Category   : Customer Escalation
Query      : Why does Escalation - Repeated Service Failures occur?
================================================================================

Detected Outcome Event:
+ Escalation - Repeated Service Failures

Identified Causal Factors:

1) Different People
   Supporting Evidence:
   - (Transcript 7034-5430-2980-5483)
     "I've already explained this to multiple different people. Each time I'm told it's fixed, but it's not."
   - (Transcript 4967-8141-1813-8658)
     "I've already explained this to four different people. Each time I'm told it's fixed, but it's not."
   - (Transcript 3631-7868-1079-2880)
     "I've already explained this to six different people. Each time I'm told it's fixed, but it's not."
```

```
================================================================================
Query ID   : 2
Category   : Customer Escalation
Query      : Why does Escalation - Repeated Service Failures occur?
================================================================================

Detected Outcome Event:
+ Escalation - Repeated Service Failures

Identified Causal Factors:

1) Different People
   Supporting Evidence:
   - (Transcript 7034-5430-2980-5483)
     "I've already explained this to multiple different people. Each time I'm told it's fixed, but it's not."
   - (Transcript 4967-8141-1813-8658)
     "I've already explained this to four different people. Each time I'm told it's fixed, but it's not."
   - (Transcript 3631-7868-1079-2880)
     "I've already explained this to six different people. Each time I'm told it's fixed, but it's not."

Causal Explanation:
The outcome 'Escalation - Repeated Service Failures' is primarily triggered by early-stage customer reports indicating issues such as Different
People. These statements reflect underlying problems that precede the event. The supporting dialogue evidence demonstrates recurring patterns th
at logically contribute to the emergence of this outcome.
================================================================================
```

Conclusion

This project presents a practical and interpretable system for performing causal analysis over large-scale conversational data. Rather than treating outcome events such as escalations, fraud investigations, or service failures as isolated labels, the system focuses on understanding **how conversational dynamics lead to these outcomes**. By operating at the level of dialogue turns and conversational patterns, the approach moves beyond surface-level text analysis and provides meaningful insight into real interaction behavior.

A key strength of the system is its emphasis on **interpretability and evidence grounding**. All causal factors are derived from explicit conversational signals and recurring patterns observed in the dataset. Each explanation produced by the system is directly traceable to specific transcripts and dialogue turns, ensuring transparency and preventing unsupported or speculative conclusions. This design choice aligns well with real-world operational requirements, where analysts and stakeholders must be able to verify and trust analytical outputs.

The use of **contrastive causal reasoning** allows the system to differentiate patterns that are merely common from those that are strongly associated with outcome events. By comparing outcome and non-outcome conversations, the system highlights conversational behaviors that meaningfully contribute to negative outcomes, such as repeated unresolved complaints, delayed agent responses, or escalation-triggering language. This makes the results actionable, as organizations can focus on addressing the identified patterns rather than reacting only after outcomes occur.

In addition to single-query analysis, the system supports **context-aware multi-turn interaction**, enabling users to explore causal explanations through follow-up questions. By explicitly maintaining session context, the system ensures consistency across interactions and avoids contradictory reasoning. This interactive capability enhances the analytical value of the system and mirrors how human analysts naturally investigate complex conversational data.

Overall, the project demonstrates that causal reasoning over conversational transcripts is achievable using structured, interpretable methods without relying on opaque models. The proposed system provides a scalable and reproducible foundation for conversational analytics and can be extended to additional domains, more complex causal modeling techniques, or real-time analysis scenarios. As conversational systems continue to grow in scale and importance, approaches like this can play a critical role in improving service quality, reducing escalations, and enabling data-driven operational decisions.